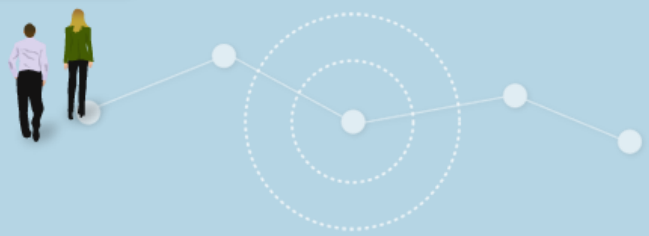


데이터베이스실습

MySQL로 배우는 데이터베이스 개론과 실습

데이터베이스 연동 파이썬 프로그래밍



데이터베이스 연동 파이썬 프로그래밍

MySQL

- **리눅스 환경에서 개발**
  - 명령 프롬프트에서 개발
  - 원격 접속을 위해 터미널 접속 프로그램 사용(putty, ftp)
  - 리눅스는 다중 계정을 가진다.

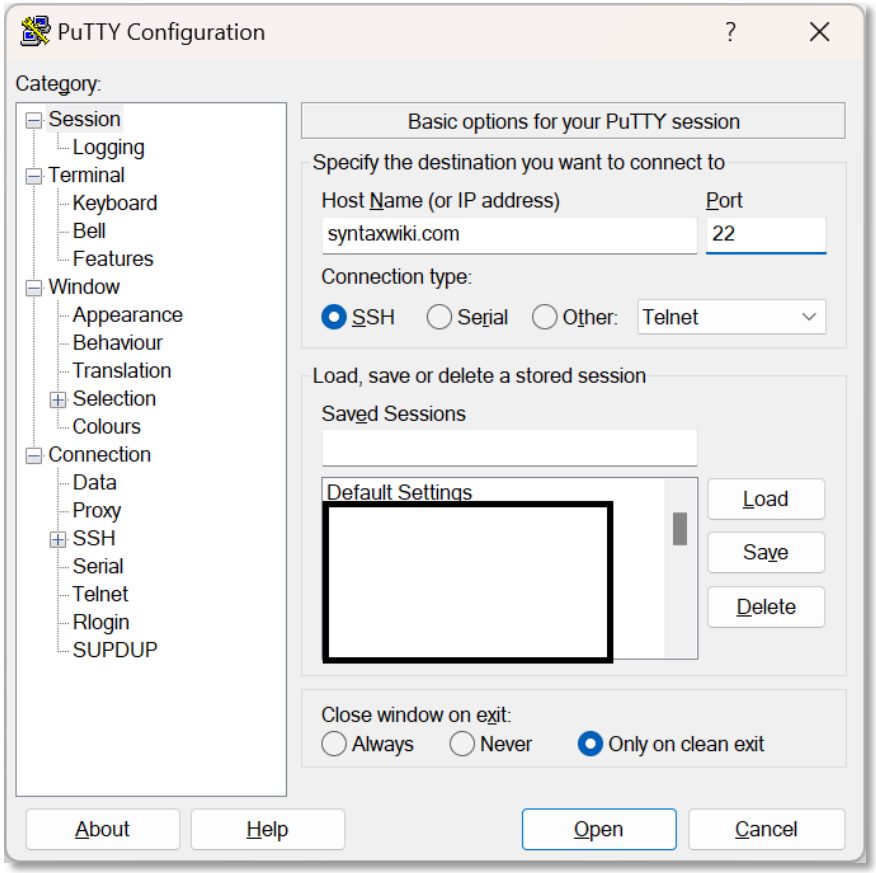
- **터미널 접속 프로그램 putty 설치**
  - https://www.putty.org/
  - 64-bit x86 Windows Installer 로 설치

- **리눅스 서버에 접속**
  - Host Name: syntaxwiki.com
  - Port: 22
  - Connection Type: SSH

데이터베이스 연동 파이썬 프로그래밍

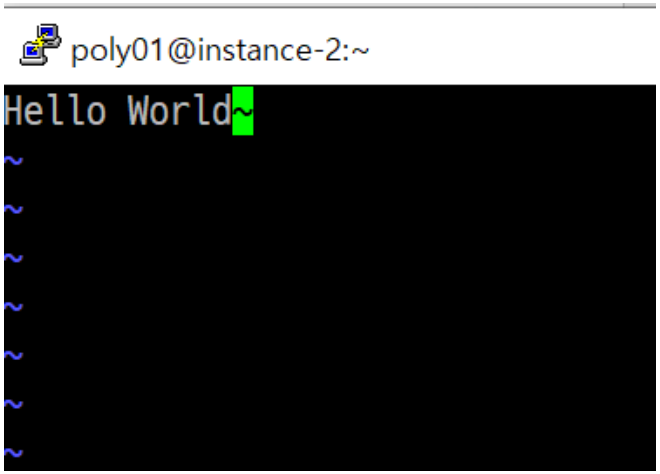
- **Putty 터미널 접속 프로그램**
  - Host Name: syntaxwiki.com
  - Port: 22
  - Connection Type: SSH

- **리눅스 계정**
  - Usename : poly01
    - (2613340001 -> 01, 학번 뒷자리 2개)
  - Password : poly2026



데이터베이스 연동 파이썬 프로그래밍

- **편집툴 VI**
  - 리눅스 기반 편집툴
  - 명령모드와 편집모드가 존재
  - 명령 프롬프트에 vi readme.md
  - 편집 모드 전환 : i
  - Hello World~ 입력
  - 명령모드 전환 : ESC
  - 파일 저장: 명령모드 상태에서 :w 입력
  - 파일 닫기: 명령모드 상태에서 :q 입력
  - 파일 저장후 닫기: 명령모드 + :wq



데이터베이스 연동 파이썬 프로그래밍

MySql 서버에 접속

- 계정: poly01(리눅스 계정과 동일, 학번 뒷자리 2개)
- 비밀번호: poly2026
- 터미널에서 mysql –upoly01 –p 입력

```
[poly01@instance-2 ~]$ mysql -upoly01 -p
Enter password:
```

- Enter password 가 표시되면 비밀번호 입력
- show databases;

```
MariaDB [(none)]> show databases;
+--------------------+
| Database           |
+--------------------+
| information_schema |
| poly01             |
+--------------------+
2 rows in set (0.006 sec)
```

- use poly01;

MySQL Client 종료

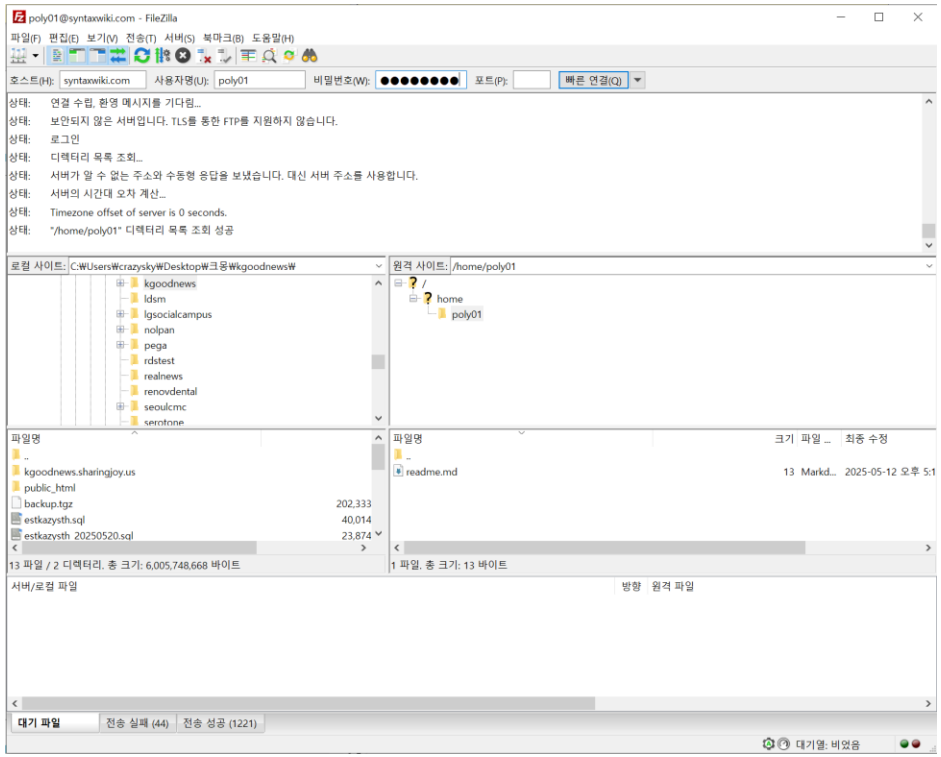
- quit;

```
MariaDB [(none)]> quit;
Bye
```

데이터베이스 연동 파이썬 프로그래밍

파일질라

- FTP 프로토콜을 사용하는 파일 관리 프로그램
- https://filezilla-project.org/
- FileZilla Client 버전 다운 및 설치



파일 업로드

- 호스트: syntaxwiki.com
- 사용자명: poly01
- 비밀번호: poly2026
- 파일 드래그로 업로드
- 실습자료실에서 practice_py.sql 다운로드
- "practice_py.sql" 업로드

데이터베이스 연동 파이썬 프로그래밍

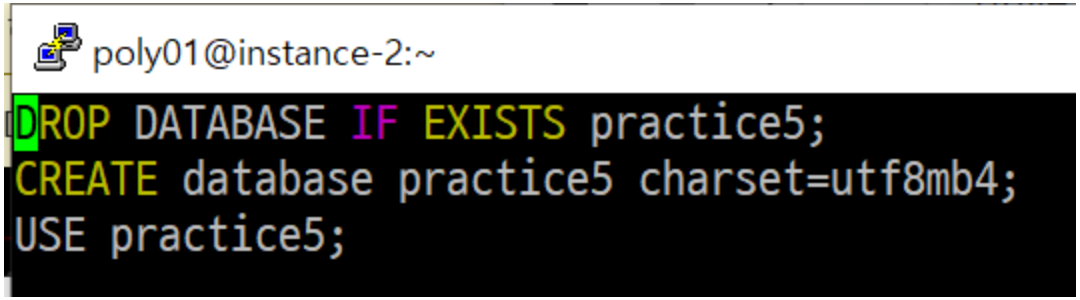
프롬프트 명령어

- 폴더 생성
  - mkdir temp
- 하위 폴더로 이동
  - cd temp
- 상위 폴더로 이동
  - cd ..
- 현재 폴더 확인
  - pwd
- 나의 계정 폴더로 이동
  - cd ~
- 현재 폴더에 소속된 폴더 및 파일 보기
  - ls

데이터베이스 연동 파이썬 프로그래밍

DB에 SQL 스크립트 입력

- vi 편집 툴로 practice_py.sql 의 사용 DB 변경
- DB Name -> poly01(학번 뒷자리 2개)



- practice5 -> poly01 로 변경
- vi 삭제 단축키
  - 명령모드 + x (커서 위치에 1개 지움)
  - 입력모드 + delete
  - 입력모드 + backspace

- 편집모드 전환 -> i

```
DROP DATABASE IF EXISTS poly01;
CREATE database poly01 charset=utf8mb4;
USE poly01;
```

데이터베이스 연동 파이썬 프로그래밍

SQL 스크립트 입력하기

- mysql –upoly01 –p poly01 < practice_py.sql
- -u -> 유저명
- -p -> 비밀번호 사용
- poly01 -> 사용할 DB Name
- < practice_py.sql -> 입력으로 사용할 스크립트 파일명

스크립트 실행 확인

- mysql –upoly01 –p
- 비밀번호 poly2026
- use poly01; //데이터베이스 지정
- show tables; //테이블 보기
- select * from Book;
- 대소문자 구별필요.

데이터베이스 연동 파이썬 프로그래밍

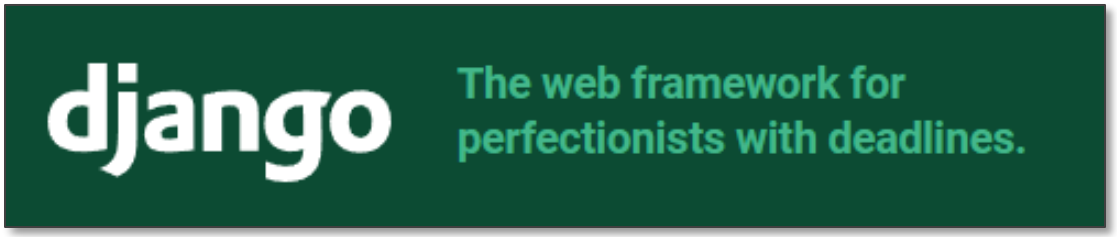
- **리눅스의 mysql(maria) 은 대소문자를 구별한다.**
- **테이블 이름을 소문자로 변경**
  - ALTER TABLE Book RENAME TO book
  - ALTER TABLE Customer RENAME TO customer;
  - ALTER TABEL Orders RENAME TO orders;

```
MariaDB [poly100]> show tables;
+------------------+
| Tables_in_poly100 |
+------------------+
| Book             |
| Customer         |
| Exaggfn          |
| Imported_Book    |
| Movie            |
| Mybook           |
| Orders           |
| Screen           |
| Theater          |
| new_movie        |
| new_screenhist   |
| screeninfo_05    |
+------------------+
12 rows in set (0.000 sec)
```

데이터베이스 연동 파이썬 프로그래밍

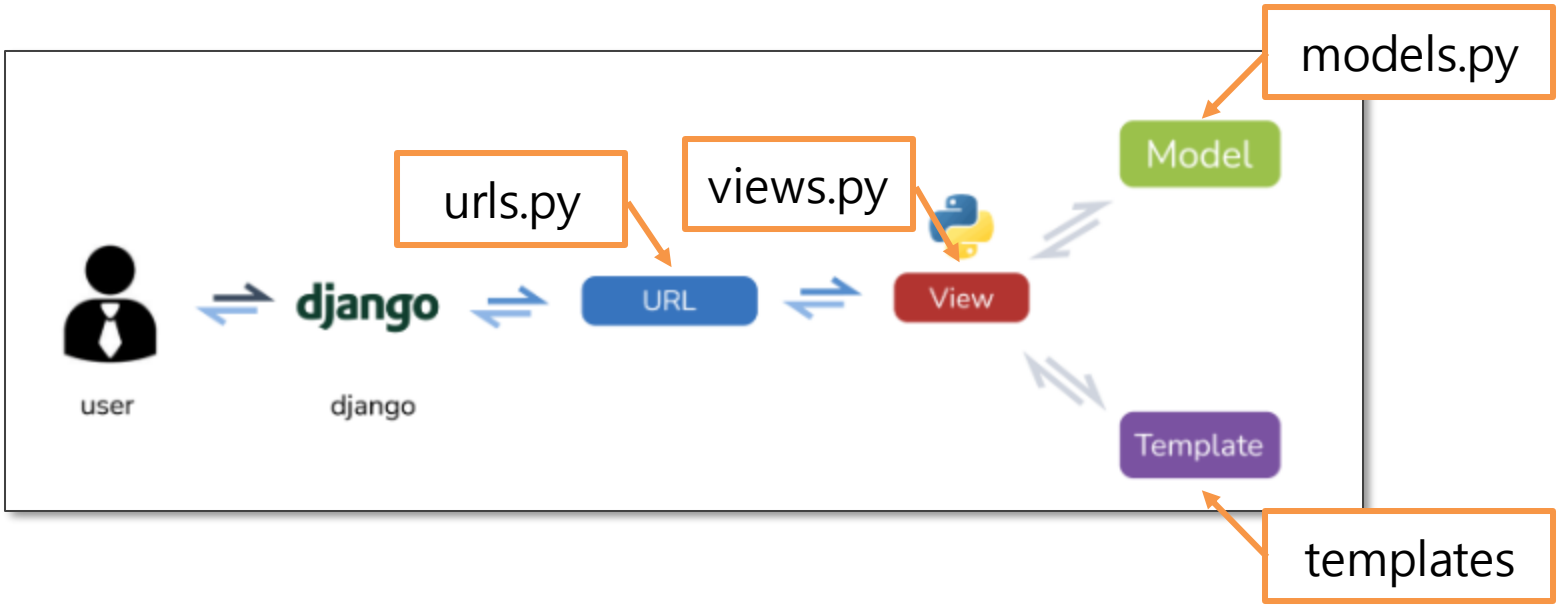
Django

- 파이썬으로 만들어진 웹 개발 프레임워크, 패키지
- 신속한 개발과 깔끔하고 실용적인 디자인을 장려하는 고급 Python 웹 프레임워크
- 숙련된 개발자가 구축한 이 패키지는 웹 개발의 번거로움을 대부분 처리하므로 바퀴를 다시 만들 필요 없음
- Django 슬로건
  - The web framework for perfectionists with deadlines.
  - 마감에 쫓기는 완벽주의자를 위한 웹 프레임워크
  - Django를 사용하면 웹 프로그램을 마감에 쫓기지 않을 정도로 빠르게 만들 수 있으면서도 완벽에 가깝다.
- 무료이며 오픈 소스
- 공식 사이트 주소
  - https://www.djangoproject.com/



데이터베이스 연동 파이썬 프로그래밍

- web은 url request와 response로 이루어져 있다.
- request는 HTML 페이지 요청이다.
- response는 HTML 페이지 응답이다.



데이터베이스 연동 파이썬 프로그래밍

- **Django 웹 프로젝트 다운로드**
  - 실습자료실에 dj_bookstore.tar.gz 파일 다운로드
  - 파일질라 이용해서 리눅스 계정 폴더에 복사
  - 압축 해제
  - tar zxvf dj_bookstore.tar.gz
  - ls 명령으로 bookstore 폴더 생성 확인
  - cd bookstore 폴더로 이동

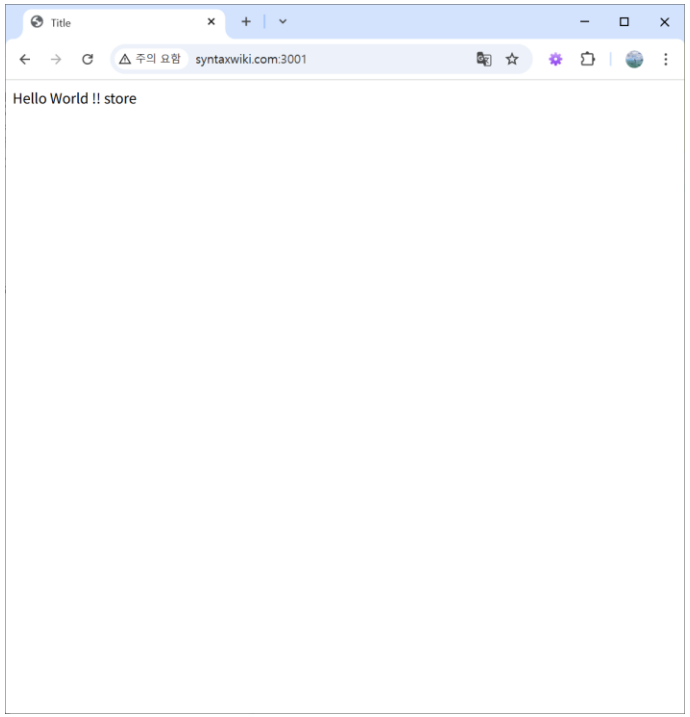
데이터베이스 연동 파이썬 프로그래밍

Django 웹 사이트 구동

- pwd 로 현재 위치가 bookstore 확인
- 임시 서버 구동
- python manage.py runserver 0:3001(30XX 학번 뒷자리)

브라우저에서 웹사이트 확인

- http://syntaxwiki.com:3001



데이터베이스 연동 파이썬 프로그래밍

- index 페이지

```
# Create your views here.
def index(request):
    context = {'app_name': 'store'}
    return render(request, 'store/index.html', context)
```

- 서버 url "/" 에 접속시 store/index.html 전송
- /store/templates/store/index.html

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <title>Title</title>
</head>
<body>
<div>Hello World !! {{app_name}}</div>
</body>
</html>
```

- 전송 내용을 Hello Django 로 변경해 보자.

데이터베이스 연동 파이썬 프로그래밍

- **MySQL 과 연동**
  - 서버의 MySQL 계정의 정보를 입력
  - 프레임워크에서 연결 함수를 지원

- **config/settings.py 수정**

```
DATABASES = {
    'default': {
        'ENGINE': 'django.db.backends.mysql'
        'NAME': 'poly01',
        'USER': 'poly01',
        'PASSWORD': 'poly2026'
    }
}
```

데이터베이스 연동 파이썬 프로그래밍

- **Django Model**
  - 데이터베이스의 테이블과 연동
  - 테이블을 형태를 정의

- **store/models.py**

```
class Book(models.Model):
    id = models.AutoField(primary_key=True)
    bookname = models.CharField(max_length=40, null=True)
    publisher = models.CharField(max_length=40, null=True)
    price = models.IntegerField(null=True)
    class Meta:
        db_table='book'
```

데이터베이스 연동 파이썬 프로그래밍

- **Book Model**

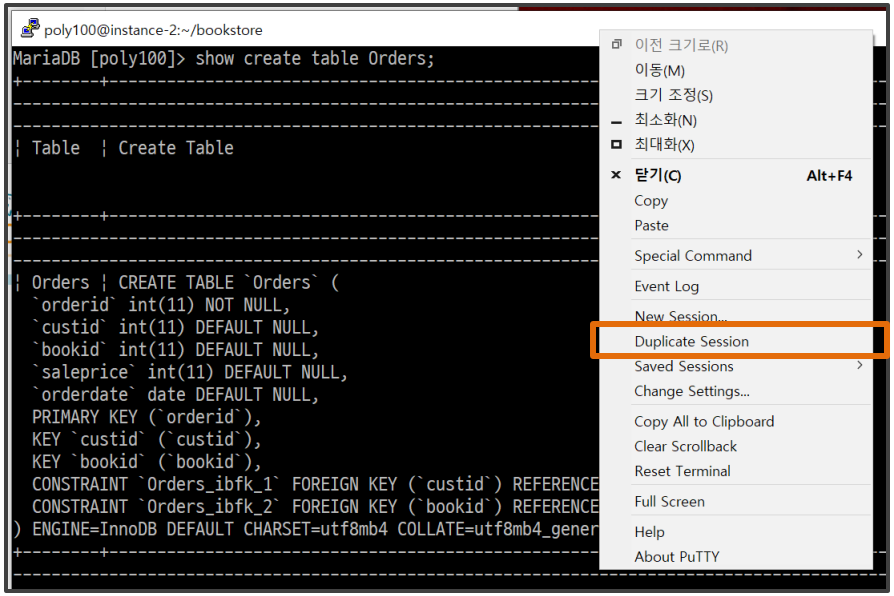
id	int(11), auto_increment	AutoField
bookname	varchar(40)	CharField
publisher	varchar(40)	CharField
price	int(11)	IntegerField

- **테이블 필드명 불일치**
  - bookid 필드를 id 로 변경
  - ALTER TABLE book CHANGE bookid id INT AUTO_INCREMENT;

데이터베이스 연동 파이썬 프로그래밍

Putty 터미널 중복 연결

- 터미널 창 상단의 헤더 부분에서 마우스 우클릭
- 메뉴에서 Duplicate Session 선택
- 터미널 로그인
- 여러개의 터미널을 사용할 수 있다.



리눅스 계정

- Usename : poly01
  - (2613340001 -> 01, 학번 뒷자리 2개)
- Password : poly2026

데이터베이스 연동 파이썬 프로그래밍

Book의 bookid 를 id로 변경

```
MariaDB [poly100]> desc book;
+-----------+-------------+------+-----+---------+-------+
| Field     | Type        | Null | Key | Default | Extra |
+-----------+-------------+------+-----+---------+-------+
| bookid    | int(11)     | NO   | PRI | NULL    |       |
| bookname  | varchar(40) | YES  |     | NULL    |       |
| publisher | varchar(40) | YES  |     | NULL    |       |
| price     | int(11)     | YES  |     | NULL    |       |
+-----------+-------------+------+-----+---------+-------+
4 rows in set (0.001 sec)
```

- ALTER TABLE book CHANGE bookid id INT AUTO_INCREMENT;

```
MariaDB [poly100]> alter table book change bookid id int auto_increment;
ERROR 1846 (0A000): ALGORITHM=COPY is not supported. Reason: Columns participating in a foreign key are renamed. Try ALGORITHM=INPLACE
```

- 에러 발생 -> orders 테이블의 Foreignkey 지정 문제
- Foreignkey 삭제 후 다시 시도 / show create table orders;

```
| orders | CREATE TABLE `orders` (
  `orderid` int(11) NOT NULL,
  `custid` int(11) DEFAULT NULL,
  `bookid` int(11) DEFAULT NULL,
  `saleprice` int(11) DEFAULT NULL,
  `orderdate` date DEFAULT NULL,
  PRIMARY KEY (`orderid`),
  KEY `custid` (`custid`),
  CONSTRAINT `orders_ibfk_1` FOREIGN KEY (`custid`) REFERENCES `customer` (`custid`)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_general_ci |
```

- ALTER TABLE orders DROP FOREIGN KEY orders_ibfk_1;

데이터베이스 연동 파이썬 프로그래밍

book 테이블에 record 추가

- book 테이블에 INSERT 문을 사용해서 새로운 데이터를 추가
- 브라우저에서 INSERT 한 내용이 표시되는지 확인

insert book record 쿼리

- INSERT INTO book VALUES (null, '데이터베이스', '폴리텍', 20000);
- INSERT INTO book SET bookname='파이썬', publisher='폴리텍', price=30000;

booklist 페이지에서 확인

- http://syntaxwiki.com:3001/booklist
- 추가된 도서 이름 확인



데이터베이스 연동 파이썬 프로그래밍

- **BookList**
  - Book.objects.all()
  - book 테이블의 모든 record 를 가져오는 함수
- **ORM 방식 (Object-Relational Mapping)**
  - 내부에서 SQL 쿼리를 자동 생성한다.
  - Object(instance) 와 관계형 데이터를 매핑(연결)한다.

```
def booklist(request):
    queryset = Book.objects.all()
    context = {'books': queryset}
    return render(request, 'store/booklist.html', context)
```

- **Raw Query 방식**

```
    with connection.cursor() as cursor:
        cursor.execute("SELECT * FROM book")
        columns = [col[0] for col in cursor.description]
        queryset = [dict(zip(columns, row)) for row in cursor.fetchall()]
```

Template

- PHP 와 Django 코드 비교

booklist.php

```php
<html>
<head>
<meta http-equiv="Content-Type" content="text/html; charset=utf-8" />
<link href="book.css" rel="stylesheet">
</head>
<body>
<h2><blockquote> 마당서점 도서목록 </blockquote></h2>
<?php
    $conn = mysqli_connect('localhost', 'madang', 'madang', 'practice3');
    if (!$conn){
        echo 'Connection Error';
        echo mysqli_connect_error();
        exit();
    }

    $sql = "SELECT * FROM Book";
    $result = mysqli_query($conn, $sql);

    echo '<table class="styled-table"><thead><tr><td>BOOKNAME</td><td>PUBLISHER</td><td>PRICE</td></tr></thead><tbody>';

    while($row = mysqli_fetch_array($result)) {
        echo ('<tr class="active-row"><td><b>
            <a href="bookview.php?id='.$row['bookid'].'">'
            .$row['bookname'].'</a></b></td><td>'
            .$row['publisher'].' </td><td> '
            .$row['price'].'</td></tr>');
    }
    echo '</tbody></table><p>';

?>
<a href="./insertbook.php" class="w-btn">책등록</a>
</body>
</html>
```

VS

url.py, views.py, booklist.html

```
<html>
<head>
<meta http
<link href
</head>
<body>
<h2><block
</body>
    <table class="styled-table">
        <thead>
        <tr>
            <td>BOOKNAME</td>
            <td>PUBLISHER</td>
            <td>PRICE</td>
        </tr>
        </thead>
        <tbody>
            {% for row in books %}
            <tr class="active-row">
                <td><b><a href="">{{ row.bookname }}</a></b></td>
    </tabl
</html>
```

```
urlpatterns = [
    path("", views.index, name="index"),
    path('booklistbycursor', views.booklistbycursor, name='booklistbycursor'),
    path('booklistbynametuple', views.booklistbynametuple, name='booklistbynametuple'),
    path('booklist', views.booklist, name='booklist'),
    path('bookview', views.bookview, name='bookview'),
    path('postbook', views.postbook, name='postbook'),
    path('book', views.book, name='book')
]
```

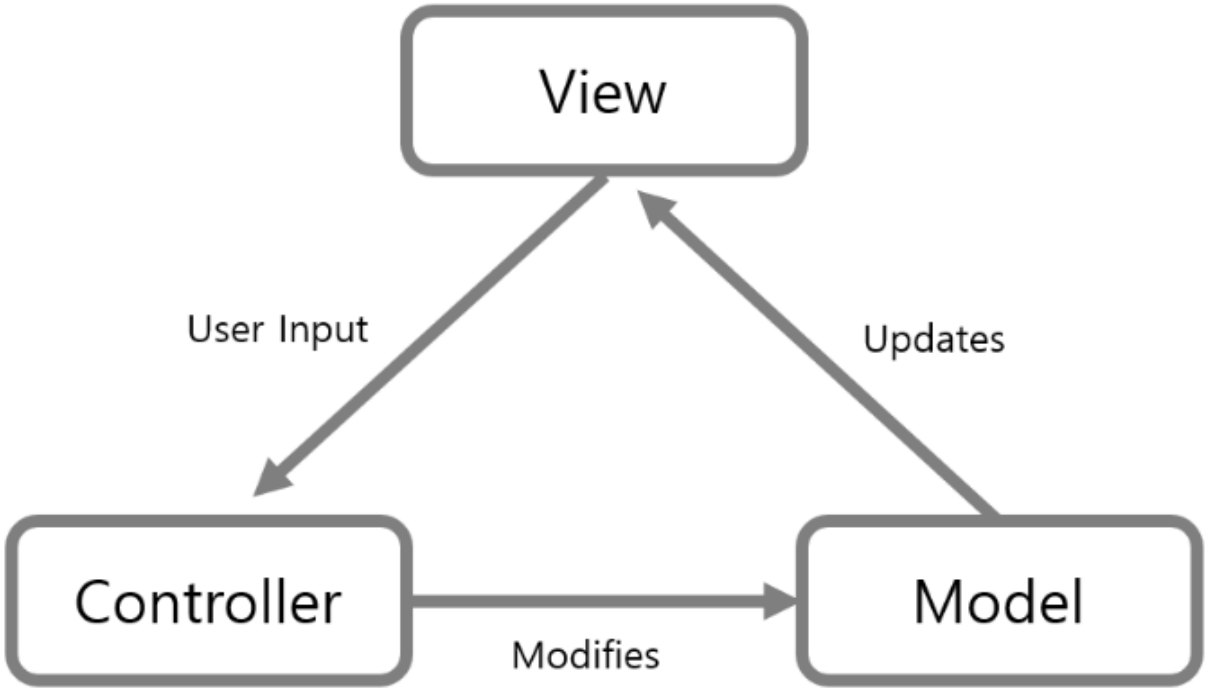
```python
def book(request):
    results = []
    with connection.cursor() as cursor:
        cursor.execute("SELECT * FROM book")
        queryset = namedtuplefetchall(cursor)

    for row in queryset:
        results.append(f'{row.bookname}')
        results.append('<br>')

    context = {'books': queryset}
    return render(request, 'polls/booklist.html', context)
```

Template

- **PHP은 파일 하나에 출력과 로직이 모두 포함되어 있다.**
- **Django는 MVC(MVT) 패턴 방식**
  - 출력과 로직이 분리되어 있다.



데이터베이스 연동 파이썬 프로그래밍

- **book 테이블에서 가격 수정**
  - 데이터베이스 책의 가격을 40000원으로 수정

- **update book record 쿼리**
  - SELECT * FROM book WHERE booname like '%데이터%';
  - ID 확인
  - UPDATE book SET price = 40000 WHERE id = ;

- **booklist 페이지에서 확인**
  - http://syntaxwiki.com:3001/booklist
  - 도서 가격 확인



데이터베이스 연동 파이썬 프로그래밍

book 테이블에서 책 삭제

- 데이터베이스 책을 삭제

delete book record 쿼리

- SELECT * FROM book WHERE booname like '%데이터%';
- ID 확인
- DELETE FROM book WHERE id = ;

booklist 페이지에서 확인

- http://syntaxwiki.com:3001/booklist
- 도서 삭제 확인



데이터베이스 연동 파이썬 프로그래밍

Bookname 클릭시 상세페이지로 이동

- bookview/id
- boolist 페이지에서 각 record에 a 태그의 href 지정

```
{% for row in books %}
<tr class="active-row">
    <td><b><a href="{% url 'bookview' row.id %}">{{ row.bookname }}</a></b></td>
    <td>{{ row.publisher }}</td>
    <td>{{ row.price }}</td>
    <td><a href="{% url 'deletebook' row.id %}"><div class="delete-btn"></div></a></td>
</tr>
{% endfor %}
```

bookview/id URL를 처리하는 함수

- store/views.py

```
def bookview(request, id):
    book = Book.objects.get(id=id)
    context = {'book': book}
    return render(request, 'store/bookview.html', context)
```

- id 를 이용해서 SELECT * FROM book WHERE id = ; 와 같은 쿼리를 자동 생성
- 결과를 book 변수에 저장

데이터베이스 연동 파이썬 프로그래밍

bookview 의 template

- /store/templates/store/bookview.html

```
<html>
<meta http-equiv="Content-Type" content="text/html; charset=utf-8" />
<link href="/static/book.css" rel="stylesheet">
<body>
    <h2><blockquote> 마당서점 도서정보 </blockquote></h2>

    <table class="styled-table">
        <tr><td>책번호</td><td>{{ book.id }}</td></tr>
        <tr><td>책제목</td><td>{{ book.bookname }}</td></tr>
        <tr><td>출판사</td><td>{{ book.publisher }}</td></tr>
        <tr><td>가격</td><td>{{ book.price }}</td></tr>
    </table>
    <a href="{% url 'editbook' book.id %}" class="w-btn">수정</a>
    <a href="/editbook/{{ book.id }}" class="w-btn">수정#</a>
    <a href="{% url 'booklist' %}" class="w-btn">목록보기</a>
    <a href="/booklist" class="w-btn">목록보기#</a>
</body>
</html>
```

- book 객체를 테이블 태그를 사용해 보여준다.
- 수정하기 버튼에 책 수정 링크 URL이 지정되어 있다.

데이터베이스 연동 파이썬 프로그래밍

- **bookview**
  - 브라우저 URL
  - /bookview/1



POST

- 도서 등록 폼(store/postbook.html)
- HTML Form 를 POST 방식으로 Request
- Request 객체의 POST 변수에 form의 내용이 저장된다.

```
def postbook(request):
    context = {}
    if request.method == 'POST':
        result = False
        try:
            bookname = request.POST.get('bookname', '')
            publisher = request.POST.get('publisher', '')
            price = request.POST.get('price')
            book = Book(bookname=bookname, publisher=publisher, price=price)
            book.save()
            result = True
        except:
            result = False
        context = {'result': result}
        return render(request, 'store/postresult.html', context)

    return render(request, 'store/postbook.html', context)
```

POST

- 도서 등록 폼(store/postbook.html)

```
<html
<meta http-equiv="Content-Type" content="text/html; charset=utf-8" />
<link href="/static/book.css" rel="stylesheet">
<body>
<h2>
    <blockquote> 마당서점 도서 등록 </blockquote>
</h2>

<form method="POST" id="register" style="width: 310px;" action="{% url 'postbook' %}">
    {% csrf_token %}
    <input type="text" class="text" name="bookname" placeholder="Book Name"><br>
    <input type="text" class="text" name="publisher" placeholder="Publisher"></br>
    <input type="number" class="text" name="price" placeholder="Price"></br>
    <input type="number" class="text" name="stock" placeholder="Stock">
    <div>
        <a href="{% url 'booklist' %}" class="form-btn">뒤로</a>

        <input type="submit" name="register" value="입력" class="form-btn" style="float: right;">
    </div>
</form>

</body>
</html>
```

Forms

- Forms.py 파일에 class 형태로 선언
- HTML 으로 부터 전달 받은 POST 데이터를 처리
- POST 데이터의 validation 체크
- POST 데이터의 생성, 수정 자동화
- HTML 템플릿 간소화

Forms

- store/forms.py 생성

```python
from django import forms
from .models import Book

3 usages
class BookForm(forms.ModelForm):
    def __init__(self, *args, **kwargs):
        super().__init__(*args, **kwargs)
        print(self.fields)
        for name, field in self.fields.items():
            print(name)
            field.widget.attrs.update({"class": "text"})

    class Meta:
        model = Book
        fields = '__all__'
```

- model을 기반으로 form을 자동으로 생성

Forms

- store/views.py

```
def createbook(request):
    if request.method == 'POST':
        form = BookForm(request.POST)
        if form.is_valid():
            form.save()
            return redirect('booklist')
    form = BookForm()
    return render(request, 'store/createbook.html', {'form': form})
```

- store/template/store/createbook.html

```
<html>
<meta http-equiv="Content-Type" content="text/html; charset=utf-8" />
<link href="/static/book.css" rel="stylesheet">
<body>
<h2><blockquote> 마당서점 도서 등록 </blockquote></h2>

<form method="POST" id="register" style="width: 310px;" action="{% url 'createbook' %}">
    {% csrf_token %}
    {{form.as_p}}
    <div>
        <a href="{% url 'booklist' %}" class="form-btn">뒤로</a>
        <input type="submit" name="register" value="입력" class="form-btn" style="float: right;">
    </div>
</form>

</body>
</html>
```

데이터베이스 연동 파이썬 프로그래밍

editbook

- 도서의 정보를 수정하는 페이지
- 브라우저 URL
- /editbook/1
- POST로 전달된 정보를 UPDATE 쿼리로 변경한다.
- /store/views.py 에 editbook 함수

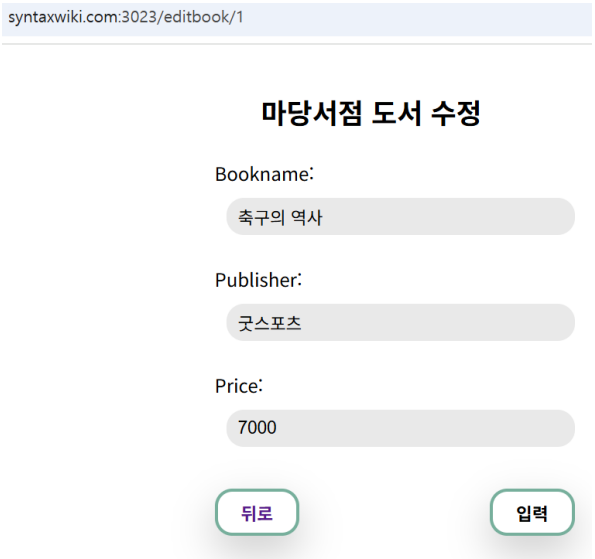
```
def editbook(request, id):
    if request.method == 'POST':
        result = '실패'
        book = Book.objects.get(id=id)
        form = BookForm(request.POST, instance=book)
        try:
            if form.is_valid():
                form.save()
                result = '성공'
            else:
                result = form.errors
        except:
            result = traceback.format_exc()
        context = {'result': result}
        return render(request, 'store/postresult.html', context)
    else:
        book = Book.objects.get(id=id)
        form = BookForm(instance=book)
        return render(request, 'store/editbook.html', {'form': form})
```

- GET Method 일때는 수정페이지 표시
- POST Method 일때는 UPDATE 쿼리 생성 및 처리

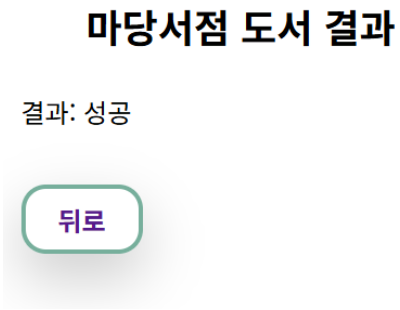
데이터베이스 연동 파이썬 프로그래밍

editbook

- URL /editbook/1
- 브라우저에서 직접 URL 입력시 GET Method 로 동작, a 태그 링크도 동일
- GET Method



- POST Method
  - form 의 submit 버튼을 눌렀을때 동작.



데이터베이스 연동 파이썬 프로그래밍

editbook template

- GET Method template
  - /store/templates/store/editbook.html

```
<html>
<meta http-equiv="Content-Type" content="text/html; charset=utf-8" />
<link href="/static/book.css" rel="stylesheet">
<body>
<h2>
    <blockquote> 마당서점 도서 수정 </blockquote>
</h2>

<form method="POST" id="register" style="width: 310px;">
    {% csrf_token %}
    <input type="hidden" name="id" value="{{form.instance.id}}">
    {{form.as_p}}
    <div>
        <a href="{% url 'booklist' %}" class="form-btn">뒤로</a>

        <input type="submit" name="register" value="입력" class="form-btn" style="float: right;">
    </div>
</form>

</body>
</html>
```

- POST Method template
  - /store/templates/store/postresult.html

```
<html>
<meta http-equiv="Content-Type" content="text/html; charset=utf-8" />
<link href="/static/book.css" rel="stylesheet">
<body>
<h2><blockquote> 마당서점 도서 결과 </blockquote></h2>

<div>결과: {{ result }}</div>
<br>
<a href="{% url 'booklist' %}" class="w-btn">뒤로</a>

</body>
</html>
```

데이터베이스 연동 파이썬 프로그래밍

deletebook

- delete url을 도서의 ID 와 함께 request

```
{% for row in books %}
<tr class="active-row">
    <td><b><a href="{% url 'bookview' row.id %}">{{ row.bookname }}</a></b></td>
    <td>{{ row.publisher }}</td>
    <td>{{ row.price }}</td>
    <td><a href="{% url 'deletebook' row.id %}"><div class="delete-btn"></div></a></td>
</tr>
{% endfor %}
```

- 파이썬 도서 삭제
  - URL: deletebook/12 -> Function: /store/views.py deletebook 함수와 연결

```
def deletebook(request, id):
    if request.method == 'POST':
        try:
            book = Book.objects.get(id=id)
            book.delete()
        except:
            result = traceback.format_exc()
        context = {'result': result}
        return render(request, 'store/postresult.html', context)
    else:
        book = Book.objects.get(id=id)
        return render(request, 'store/deletebook.html', {'book': book})
```

데이터베이스 연동 파이썬 프로그래밍

deletebook page

/deletebook/12

마당서점 도서 삭제

=

책번호	12
책제목	파이썬
출판사	폴리텍
가격	30000

취소 삭제

데이터베이스 연동 파이썬 프로그래밍

deletebook template

- /store/templates/store/deletebook.html

```
<html
<meta http-equiv="Content-Type" content="text/html; charset=utf-8" />
<link href="/static/book.css" rel="stylesheet">
<body>
    <div class="container" style="width: 400px;">
    <h2><blockquote> 마당서점 도서 삭제 </blockquote></h2>

    <table class="styled-table">
        <tr><td>책번호</td><td>{{ book.id }}</td></tr>
        <tr><td>책제목</td><td>{{ book.bookname }}</td></tr>
        <tr><td>출판사</td><td>{{ book.publisher }}</td></tr>
        <tr><td>가격</td><td>{{ book.price }}</td></tr>=
    </table>

    <form method="POST">
        {% csrf_token %}
        <a href="{% url 'booklist' %}" class="w-btn">취소</a>
        <input type="submit" name="register" value="삭제" class="form-btn" style="float: right;">
    </form>

    </div>
</body>
</html>
```

- 삭제 버튼 submit 클릭시 POST로 request 가 일어나 삭제 쿼리가 실행 됨

데이터베이스 연동 파이썬 프로그래밍

deletebook template

- /store/templates/store/deletebook.html

```
<html
<meta http-equiv="Content-Type" content="text/html; charset=utf-8" />
<link href="/static/book.css" rel="stylesheet">
<body>
    <div class="container" style="width: 400px;">
    <h2><blockquote> 마당서점 도서 삭제 </blockquote></h2>

    <table class="styled-table">
        <tr><td>책번호</td><td>{{ book.id }}</td></tr>
        <tr><td>책제목</td><td>{{ book.bookname }}</td></tr>
        <tr><td>출판사</td><td>{{ book.publisher }}</td></tr>
        <tr><td>가격</td><td>{{ book.price }}</td></tr>=
    </table>

    <form method="POST">
        {% csrf_token %}
        <a href="{% url 'booklist' %}" class="w-btn">취소</a>
        <input type="submit" name="register" value="삭제" class="form-btn" style="float: right;">
    </form>

    </div>
</body>
</html>
```

- 삭제 버튼 submit 클릭시 POST로 request 가 일어나 삭제 쿼리가 실행 됨

데이터베이스 연동 파이썬 프로그래밍

deletebook 오류 수정

- error 코드를 읽어 보고 이유를 추정해보자.

UnboundLocalError at /deletebook/8

local variable 'result' referenced before assignment

Request Method:	POST
Request URL:	http://syntaxwiki.com:3023/deletebook/8
Django Version:	4.2.15
Exception Type:	UnboundLocalError
Exception Value:	local variable 'result' referenced before assignment
Exception Location:	/home/tintin25/polytech/bookstore/store/views.py, line 100, in deletebook
Raised during:	store.views.deletebook
Python Executable:	/usr/bin/python
Python Version:	3.9.21
Python Path:	['/home/tintin25/polytech/bookstore', '/usr/lib64/python39.zip', '/usr/lib64/python3.9', '/usr/lib64/python3.9/lib-dynload', '/home/tintin25/.local/lib/python3.9/site-packages', '/usr/local/lib64/python3.9/site-packages', '/usr/local/lib/python3.9/site-packages', '/usr/lib64/python3.9/site-packages', '/usr/lib/python3.9/site-packages']
Server time:	Sun, 18 May 2025 04:24:18 +0000

- result 변수를 할당하지 않고 사용
  - result 변수의 기본값을 설정하자.

데이터베이스 연동 파이썬 프로그래밍

- **고객 관련 페이지를 작성하자.**
  - 도서와 관련된 함수를 복사해서 수정해보자.
  - template 은 이미 제공되어 있으므로 views.py 에 함수만 작성하면 된다.
  - url과 function 의 연결은 /store/urls.py 에 설정되어 있다.

```
from django.urls import path
from . import views

urlpatterns = [
    path('index', views.index, name='index'),

    path('booklist', views.booklist, name='booklist'),
    path('bookview/<int:id>', views.bookview, name='bookview'),
    path('postbook', views.postbook, name='postbook'),
    path('createbook', views.createbook, name='createbook'),
    path('editbook/<int:id>', views.editbook, name='editbook'),
    path('deletebook/<int:id>', views.deletebook, name='deletebook'),

    #path('customerlist', views.customerlist, name='customerlist'),
    #path('customerview/<int:id>', views.customerview, name='customerview'),
    #path('createcustomer', views.createcustomer, name='createcustomer'),
    #path('editcustomer/<int:id>', views.editcustomer, name='editcustomer'),
    #path('deletecustomer/<int:id>', views.deletecustomer, name='deletecustomer')

    #path('orderlist', views.orderlist, name='orderlist'),
]
```

  - 함수를 정의 할때 마다 주석을 해제하자.

데이터베이스 연동 파이썬 프로그래밍

- **도서 관련 함수 복제**
  - VI 툴 복사 / 붙여넣기
  - 명령 모드에서 SHIFT + V
  - 커서로 선택영역 지정

```
def booklist(request):
    queryset = Book.objects.all()
    with connection.cursor() as cursor:
        cursor.execute("SELECT * FROM book")
        columns = [col[0] for col in cursor.description]
        queryset = [dict(zip(columns, row)) for row in cursor.fetchall()]

    print(queryset)
    context = {'books': queryset}
    return render(request, 'store/booklist.html', context)
```

  - 복사하기 -> y 단축키
  - 붙여 넣을 곳으로 커서로 이동
  - 붙여넣기 -> p 단축키
  - 편집 모드로 함수 이름 수정

데이터베이스 연동 파이썬 프로그래밍

Customer의 custid 를 id로 변경

```
MariaDB [poly100]> desc customer;
+-----------------+-------------+------+-----+---------+-------+
| Field           | Type        | Null | Key | Default | Extra |
+-----------------+-------------+------+-----+---------+-------+
| custid          | int(11)     | NO   | PRI | NULL    |       |
| name            | varchar(40) | YES  |     | NULL    |       |
| resident_number | varchar(24) | YES  |     | NULL    |       |
| address         | varchar(50) | YES  |     | NULL    |       |
| phone           | varchar(20) | YES  |     | NULL    |       |
+-----------------+-------------+------+-----+---------+-------+
5 rows in set (0.001 sec)
```

- ALTER TABLE customer CHANGE custid id INT AUTO_INCREMENT;

```
MariaDB [poly100]> alter table customer change custid id int auto_increment;
ERROR 1846 (0A000): ALGORITHM=COPY is not supported. Reason: Columns participating in a foreign key are renamed. Try ALGORIT
HM=INPLACE
```

- 에러 발생 -> orders 테이블의 Foreignkey 지정 문제
- Foreignkey 삭제 후 다시 시도

```
| orders | CREATE TABLE `orders` (
  `orderid` int(11) NOT NULL,
  `custid` int(11) DEFAULT NULL,
  `bookid` int(11) DEFAULT NULL,
  `saleprice` int(11) DEFAULT NULL,
  `orderdate` date DEFAULT NULL,
  PRIMARY KEY (`orderid`),
  KEY `custid` (`custid`),
  CONSTRAINT `orders_ibfk_1` FOREIGN KEY (`custid`) REFERENCES `customer` (`custid`)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_general_ci |
```

- ALTER TABLE orders DROP FOREIGN KEY orders_ibfk_1;

데이터베이스 연동 파이썬 프로그래밍

- **URL /customerlist**
  - customerlist
  - createcustomer
  - customerview
  - deletecustomer

- **forms.py 에 CustomerForm 클래스 정의**

```python
from django import forms
from .models import Book, Customer

class BookForm(forms.ModelForm):

    def __init__(self, *args, **kwargs):
        super().__init__(*args, **kwargs)
        print(self.fields)
        for name, field in self.fields.items():
            print(name)
            field.widget.attrs.update({"class": "text"})

    class Meta:
        model = Book
        fields = '__all__'

class CustomerForm(forms.ModelForm):

    def __init__(self, *args, **kwargs):
        super().__init__(*args, **kwargs)
        print(self.fields)
        for name, field in self.fields.items():
            print(name)
            field.widget.attrs.update({"class": "text"})

    class Meta:
        model = Customer
        fields = '__all__'
```

데이터베이스 연동 파이썬 프로그래밍

- customer list 에서 필요한 함수

마당서점 고객목록

이름	주소	연락처	이메일	DELETE
박지성	영국 맨체스타	000-5000-0001		
김연아	대한민국 서울	000-6000-0001		
장미란	대한민국 강원도	000-7000-0001		
추신수	미국 클리블랜드	000-8000-0001		
박세리	대한민국 대전	None		

고객등록